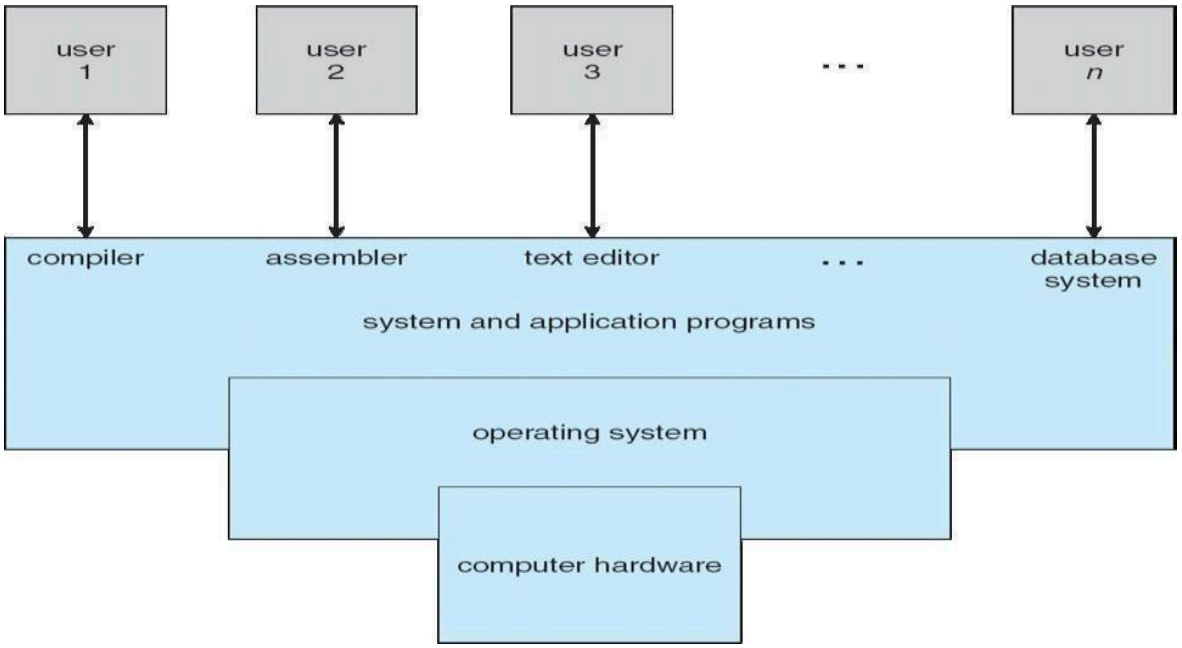
MODULE 1

1. Introduction: What Operating Systems Do

Operating System Architecture



1. Introduction: Definition of Operating System Architecture

An **Operating System (OS)** is a **system software** that acts as an **intermediary between the user and the computer hardware**. It manages the hardware resources such as the **CPU**, **memory**, **input/output devices**, and **storage**, while also providing an environment in which user applications can run smoothly and efficiently.

The **architecture of an operating system** defines the **internal structure** of the OS and explains **how different components interact** with each other to perform various tasks. Just like a building has a blueprint that determines how different rooms and utilities are connected, an OS architecture is the blueprint that determines how the operating system is designed and how it operates.

The main goal of OS architecture is to ensure that **hardware resources are used efficiently**, applications run **smoothly**, and the **user has a simple interface** to interact with the system. Every modern OS—such as **Windows**, **macOS**, **Linux**, **Android**, and **iOS**—is built upon a well-defined architectural structure.

Main Functions of an Operating System

An operating system plays a critical role in making a computer system work smoothly and efficiently. One of its most important jobs is **allocating CPU time**. Since many programs may want to run at the same time, the operating system decides which process will use the CPU and for how long. This process is known as CPU scheduling. By managing CPU time properly, the OS ensures that every active program gets a fair share of processing power and that the system remains fast and responsive. For example, when you open multiple applications like a browser, music player, and document editor, the OS rapidly switches between them, making it seem like everything is running at the same time.

Another essential function is **managing the memory**. Memory is a limited and valuable resource. The OS keeps track of which part of memory is being used by which program and ensures that no program interferes with another. It allocates memory to programs when they start and frees it up when they finish. The OS also uses techniques like virtual memory and paging to make efficient use of physical RAM, allowing larger programs to run smoothly even if the computer's physical memory is limited.

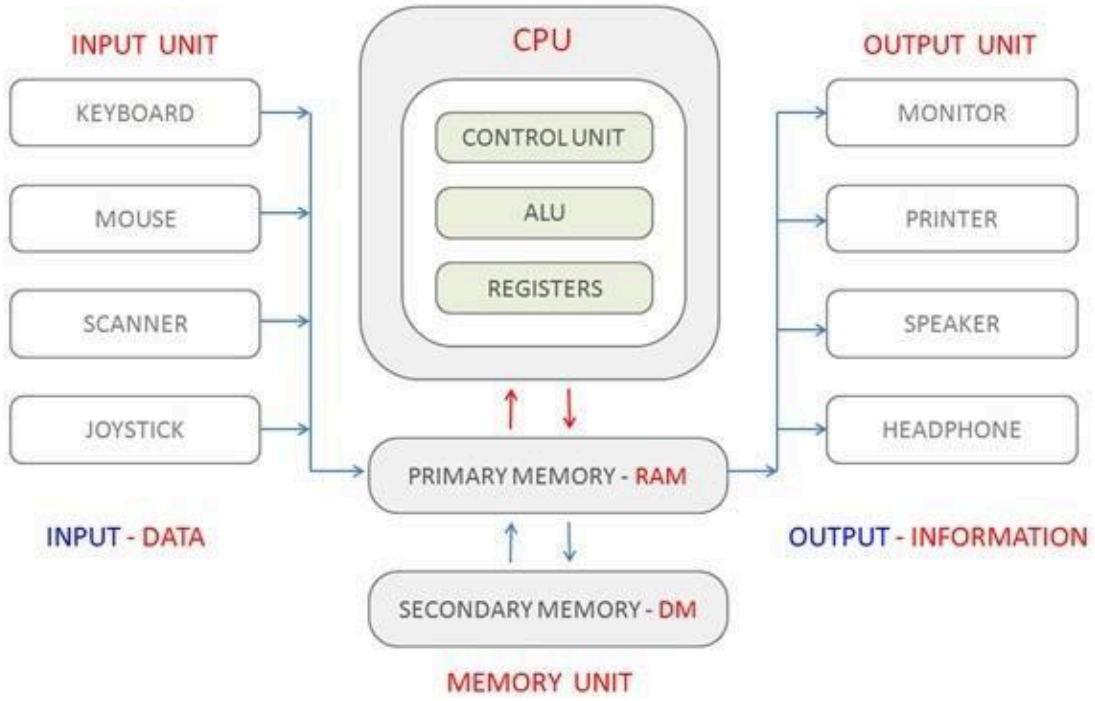
The operating system also **controls input and output (I/O) devices**. These devices include keyboards, mice, monitors, printers, hard drives, and USB devices. Instead of allowing programs to communicate with these devices directly (which could be risky or inefficient), the OS provides a controlled interface. It manages data transfer between programs and devices, ensuring everything runs smoothly. For example, when you print a document, the OS sends the data to the printer driver, which then controls the printing process without the user needing to handle any technical details.

Additionally, the operating system is responsible for **providing security and a user interface**. It protects the system from unauthorized access by using passwords, user accounts, file permissions, and security policies. It ensures that each user can only access the resources they are allowed to use, helping to keep the system safe from attacks or accidental damage. Along with security, the OS offers a **user interface**, which allows people to interact with the computer easily. This can be a CommandLine Interface (CLI) for technical users or a Graphical User Interface (GUI) for general users. A GUI provides windows, icons, buttons, and menus that make the computer simple and friendly to use.

Operating systems (OS) bridge users and computer hardware. They:

- Provide a safe and efficient environment for executing programs
- Allocate system resources (CPU, memory, I/O)
- Ensure system security and multitasking
- Offer interfaces (CLI, GUI)

2. Computer Architecture



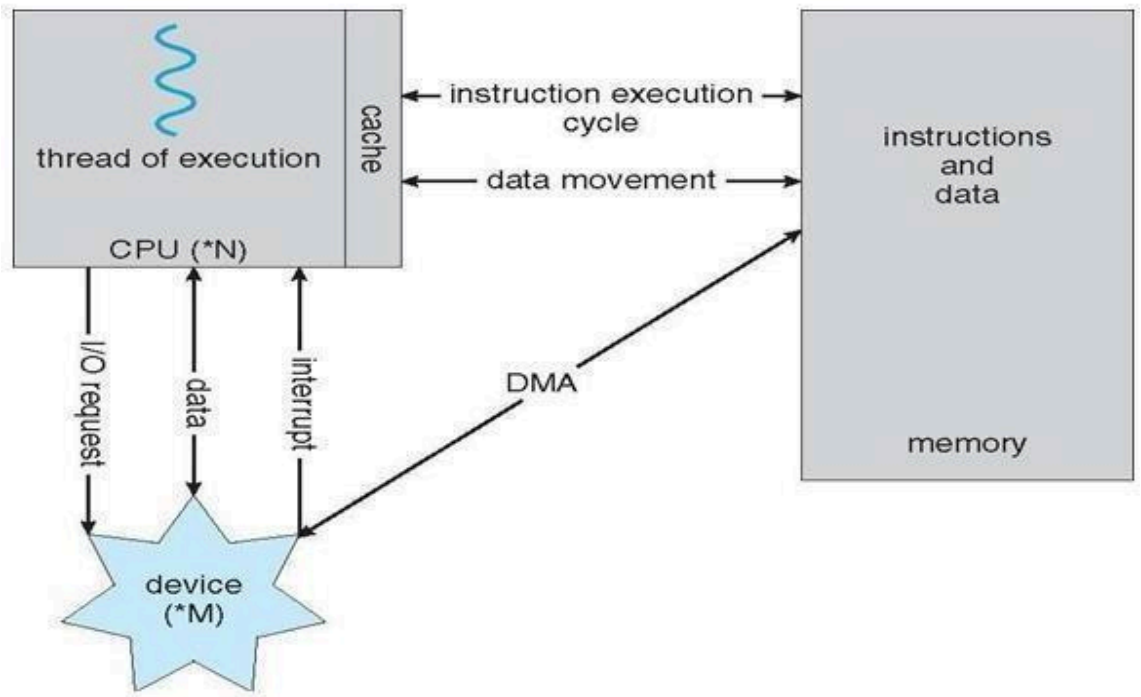
A typical computer system comprises:

- Input Unit: Devices for user data entry (keyboard, mouse)
- Output Unit: Devices for result display (monitor, printer)
- Memory Unit: Primary (RAM), secondary (HDD/SSD)
- Arithmetic Logic Unit (ALU): Performs calculations
- Control Unit: Directs operation
- CPU: ALU + Control Unit

Computer System Architecture

Types of Computer Systems (Based on Number of General-Purpose Processors)

A computer system can be classified based on how many **general-purpose processors (CPUs)** it uses to perform computations.



1. Single Processor System

Definition

A **single processor system** is a computer system that has **only one general-purpose CPU** to execute all user programs and operating system tasks.

Explanation

- The system contains **one main CPU** that performs all computations.
- It may include **special-purpose processors**, such as:
  - Disk controllers
  - I/O controllers
  - Graphics processors (GPU)
- These special processors **do not execute user programs**; they assist the main CPU.

Working

- All instructions are executed **sequentially** by the single CPU.

- Multitasking is achieved using **time sharing**, not true parallelism.

Characteristics

- One CPU controls the entire system
- Simple architecture
- Limited performance for heavy workloads

Advantages

- Simple design
- Low cost
- Easy to manage

Disadvantages

- No parallel execution
- Low throughput
- If the CPU fails, the whole system stops

Example

- Early personal computers
- Simple embedded systems

---

2. Multiprocessor Systems

Definition

A **multiprocessor system** consists of **two or more CPUs** that share the same memory, bus, and peripheral devices to perform tasks **in parallel**.

Explanation

- Also called **parallel systems**
- CPUs are **tightly coupled**
- All processors work under a **single operating system**
- Tasks can be divided among processors

Working

- Multiple CPUs execute instructions **simultaneously**
- Processes or threads are distributed among processors
- Shared memory allows fast communication

Characteristics

- Shared main memory
- Shared I/O devices
- Single OS image

Advantages

1. **Increased Throughput**
   More work is completed in less time.
2. **Economy of Scale**
   Shared memory and devices reduce overall cost.
3. **Increased Reliability**
   Failure of one CPU does not stop the system completely.

Disadvantages

- Complex hardware design
- Synchronization issues
- Higher initial cost

Types of Multiprocessor Systems

a) Symmetric Multiprocessing (SMP)

Definition

In **SMP**, all processors are **equal**, and each processor can execute **any task**, including OS functions.

Explanation

- Every CPU runs the same OS
- No master–slave relationship
- Load is dynamically balanced

Working

- Processes are placed in a common queue
- Any available CPU can execute any process
- OS manages synchronization and scheduling

Advantages

- Better load balancing
- High performance
- No single point of failure

Disadvantages

- Complex OS design
- Memory contention

Example

- Modern multicore processors
- Servers and workstations

---

b) Asymmetric Multiprocessing (AMP)

Definition

In **AMP**, processors are **not equal**. One processor acts as a **master**, and others act as **slaves**.

Explanation

- Master CPU controls the system
- Slave CPUs execute tasks assigned by the master
- Each CPU may have a specific function

Working

- Master assigns work to slaves
- Slaves execute only assigned tasks
- Communication is controlled by the master

Advantages

- Simple design
- Easy to implement
- Useful for specialized applications

Disadvantages

- Master CPU is a bottleneck
- Failure of master stops the system

- Poor load balancing

Example

- Embedded systems
- Real-time systems

---

3. Clustered Systems

Definition

A **clustered system** consists of **two or more independent computer systems** connected together to work as a **single system**.

Explanation

- Each system has:
    - Its own CPU
    - Own memory
    - Own operating system
- Systems are **loosely coupled**
- Connected via high-speed network

Working

- Tasks are distributed among systems
- If one system fails, another takes over
- Often used for **high availability**

Characteristics

- Independent nodes

- Network-based communication
- Shared storage is common

Types of Clustering

a) Asymmetric Clustering

- One system is in **hot-standby mode**
- Active system runs applications
- Standby system takes over if failure occurs

b) Symmetric Clustering

- All systems run applications
- Load is balanced
- Higher performance and availability

Advantages

- High availability
- Scalability
- Fault tolerance

Disadvantages

- Complex setup
- Network dependency
- Higher cost

Example

- Cloud computing systems
- Database servers

- High-performance computing (HPC)

Categorized roughly according to the number of general-purpose processors used.

Single-Processor Systems –

- The variety of single-processor systems range from PDAs through mainframes. On a single-processor system, there is one main CPU capable of executing instructions from user processes. It contains special-purpose processors, in the form of device-specific processors, for devices such as disk, keyboard, and graphics controllers.
- All special-purpose processors run limited instructions and do not run user processes. These aremanaged by the operating system; the operating system sends them information about their next task and monitors their status.
- For example, a disk-controller processor, implements its own disk queue and scheduling algorithm, thus reducing the task of main CPU. Special processors in the keyboard, convertsthe keystrokes into codes to be sent to the CPU.
- The use of special-purpose microprocessors is common and does not turn a single- processor system into a multiprocessor. If there is only one general-purpose CPU, then the system is a single-processor system.

Multi -Processor Systems (parallel systems or **tightly coupled systems)**
Systems that have two or more processors in close communication, sharing the computer bus, theclock, memory, and peripheral devices are the multiprocessor systems.

Multiprocessor systems have three main advantages:

1. *Increased throughput* - In multiprocessor system, as there are multiple processors execution of different programs take place simultaneously. Even if the number of processors is increased the performance cannot be simultaneously increased. This is due to the overhead incurred in keeping all the parts working correctly and also due to the competition for the shared resources. The speed-up ratio with N processors is not N, rather, it is less than N. Thus the speed of the system is not has expected.

2. *Economy of scale* - Multiprocessor systems can cost less than equivalent number of many single-processor systems. As the multiprocessor systems share peripherals, mass storage, and power supplies, the cost of implementing this system is economical. If several processes are working on the same data, the data can also be shared among them.

3. *Increased reliability*- In multiprocessor systems functions are shared among several processors. If one processor fails, the system is not halted, it only slows down. The job of the failed processor is taken up, by other processors.

   Two techniques to maintain 'Increased Reliability' - graceful degradation & fault tolerant

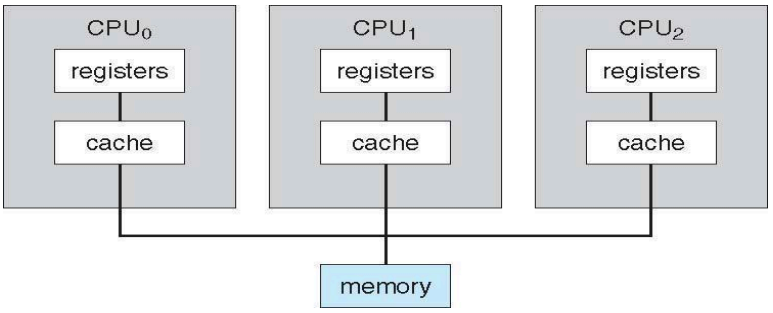
1. **Graceful degradation** – As there are multiple processors when one processor fails otherprocess will take up its work and the system go down slowly.
2. **Fault tolerant** – When one processor fails, its operations are stopped, the system failureis then detected, diagnosed, and corrected.

Different types of multiprocessor systems

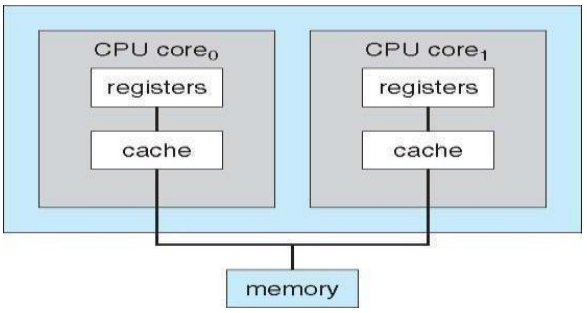
1. Asymmetric multiprocessing
2. Symmetric multiprocessing

1) *Asymmetric multiprocessing* – **(Master/Slave architecture)** Here each processor is assigned a specific task, by the master processor. A master processor controls the other processors in the system. It schedules and allocates work to the slave processors.

2) *Symmetric multiprocessing (SMP)* – All the processors are considered peers.
There is no master-slave relationship. All the processors have their own registers and CPU, only memory is shared.



The benefit of this model is that many processes can run simultaneously. N processes can run if there are N CPUs—without causing a significant deterioration of performance. Operating systems like Windows, Windows XP, Mac OS X, and Linux—now provide support for SMP. A recent trend in CPU design is to include multiple compute **cores** on a single chip. The communication between processors within a chip is faster than communication between two single processors.



Clustered Systems

Clustered systems are two or more individual systems connected together via a network and sharing software resources. Clustering provides high availability of resources and services. The service will continue even if one or more systems in the cluster fail. High availability is generally obtained by storing a copy of files (s/w resources) in the system.

There are two types of Clustered systems – **asymmetric** and **symmetric**

1. *Asymmetric clustering* – one system is in **hot-standby mode** while the others are running the applications. The hot-standby host machine does nothing but monitor the activeserver. If that server fails, the hot-standby host becomes the active server.

2. *Symmetric clustering* – two or more systems are running applications, and are monitoring each other. This mode is more efficient, as it uses all of the available hardware. If any system fails, its job is taken up by the monitoring system.

Other forms of clusters include parallel clusters and clustering over a wide-area network (WAN). Parallel clusters allow multiple hosts to access the same data on the shared storage. Clustertechnology is changing rapidly with the help of **SAN (storage-area networks)**. Using SANresources can be shared with dozens of systems in a cluster, that are separated by miles.

3. Operating System Structure or

Components of OS Architecture

The structure of an operating system is made up of several key components that work together to manage resources and provide services. The major components include:

- **Kernel**
- **System Calls**
- **Shell/User Interface**
- **Device Drivers**
- **File System**

Each of these components has a specific role to play in ensuring the smooth functioning of the operating system.

2.1 Kernel

The **kernel** is the **core component** of an operating system. It has full control over the hardware and is responsible for managing all major activities of the computer system. The kernel acts as a **bridge between software and hardware**, ensuring that software applications can use the hardware without needing to understand its complex details.

Key responsibilities of the kernel include:

- **Process Management:** Scheduling processes, switching between them, and handling multitasking.
- **Memory Management:** Allocating and freeing memory for processes.

- **Device Management:** Communicating with hardware devices through device drivers.
- **System Calls Handling:** Managing communication between user programs and the system.

Since the kernel operates at the **lowest level of the operating system**, it runs in **privileged mode** (also known as kernel mode), meaning it has unrestricted access to system resources.

2.2 System Calls

System calls are the **interface between user programs and the kernel**. They provide a way for userlevel applications to **request services** from the operating system.

For example, when you open a file, your application does not directly interact with the disk. Instead, it makes a **system call** to the kernel, which then performs the necessary operations to access the file.

Common types of system calls include:

- **Process control:** Creating, terminating, and managing processes.
- **File manipulation:** Reading, writing, opening, and closing files.
- **Device manipulation:** Communicating with hardware devices.
- **Information maintenance:** Getting system time, system information, etc.
- **Communication :** Sending and receiving messages between processes.

System calls make it easier for user applications to perform complex operations without directly accessing hardware. This ensures **security**, **stability**, and **hardware independence**.

2.3 Shell / User Interface

The **shell** is the **outermost layer of the operating system** that interacts directly with the user. It acts as a **translator** between the user and the kernel. The shell takes commands from the user, interprets them, and sends the corresponding requests to the kernel.

There are two main types of user interfaces:

- **Command-Line Interface (CLI):** In this interface, the user types text commands to interact with the system. Examples include the **Linux terminal**, **MS-DOS**, and UNIX shell. CLIs are powerful and flexible but require the user to know command syntax.
- **Graphical User Interface (GUI):** This interface uses **icons, windows, buttons, and menus**, allowing users to interact with the computer visually. Examples include **Windows OS**, **macOS**, and **Android**. GUIs are more user-friendly and widely used today.

The shell provides a **user-friendly environment** and is essential for ensuring smooth communication between the human user and the machine.

2.4 Device Drivers

Device drivers are special programs that allow the operating system to **communicate with hardware devices** such as printers, keyboards, disk drives, network cards, and monitors. Each hardware device requires its own specific driver.

The device driver acts as a **translator between the OS and the hardware**. When an application or the kernel wants to use a device, it communicates with the driver, and the driver then sends the appropriate commands to the hardware.

For example:

- A **printer driver** converts a document into a format the printer understands.
- A **graphics driver** enables the operating system to control and display content on the monitor.

Without device drivers, the operating system would not be able to **control or use hardware effectively**. Modern operating systems come with built-in drivers for common devices and allow installation of new drivers as needed.

2.5 File System

The **file system** is the component of the operating system that **organizes and manages data on storage devices**. It provides a structured way to **store, retrieve, and manipulate files** on the computer.

When a user saves a document, the OS uses the file system to store it in a specific location on the disk. Similarly, when a user opens a file, the file system locates it and provides access.

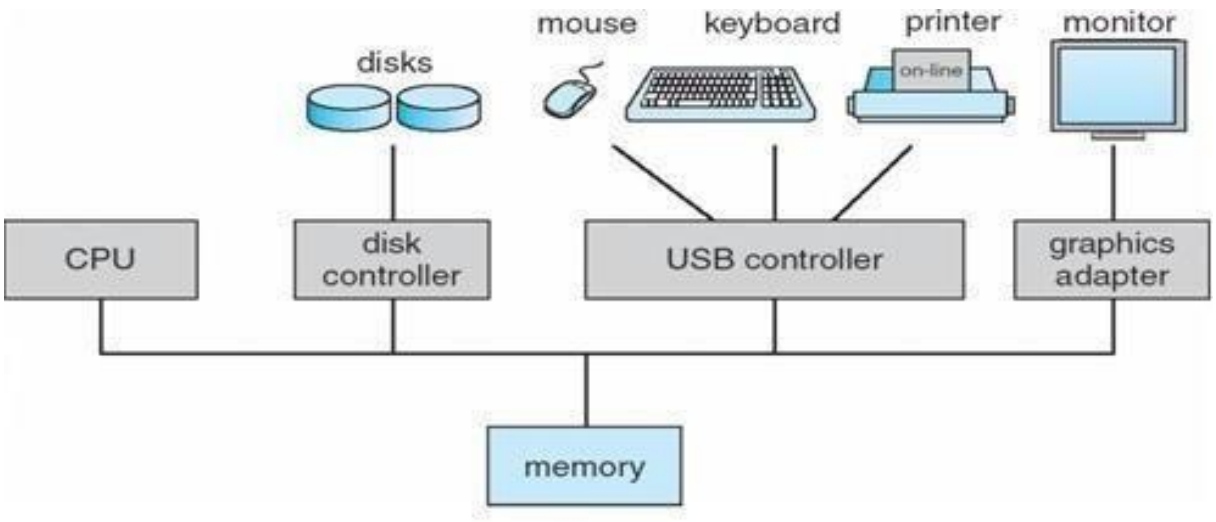
Key responsibilities of the file system include:

- **File organization:** Storing files in folders and directories.
- **File access:** Opening, reading, writing, and closing files.
- **Security and permissions:** Controlling who can access files.
- **Space management:** Keeping track of used and free space on storage devices.

Modern OS structure typically includes:

- Kernel: Core manager (processes, memory, device)
- Shell/User Interface: Communicates user requests
- Device Drivers: Peripheral handling
- File System: Data organization

4. OS Operations



- Process Management: Creating, scheduling, terminating processes
- Memory Management: Allocating/deallocating space
- File Management: Organizing and securing data
- Device Management: Interfacing peripherals
- Networking

4. OS Operations

Process management — creating, scheduling, terminating processes

Process management is the OS's responsibility for running programs: creating a process (loading an executable, allocating its initial resources), giving it CPU time according to

some scheduling policy, and cleaning up when it finishes. Creation often involves a fork() (duplicate) and exec() (replace) sequence on UNIX-like systems; termination returns an exit code and lets the parent wait() to collect it. Scheduling decides which ready process runs next (examples: round-robin gives each process a time slice, priority scheduling favors higher-priority tasks, or multi-level feedback queues adapt priorities dynamically). Example: when you run a web browser, the OS creates a browser process, the scheduler slices CPU time among the browser and other apps, and when you close the browser the OS reclaims its memory and file handles.

Memory management — allocating/deallocating space

Memory management assigns RAM to processes, isolates them from one another, and gives the illusion of more memory using techniques like virtual memory and paging. The OS maintains page tables that map a process's virtual addresses to physical frames and uses swapping or demand paging to move inactive pages to disk. Allocation/deallocation can be done by system calls (e.g., malloc() in user space backed by brk()/mmap()), and the kernel handles fragmentation, protection bits, and shared memory regions. Example: if a program requests a large array, the OS maps virtual pages for it; if physical RAM is scarce, the OS may page out seldom-used pages to disk so the active working set remains in memory.

File management — organizing and securing data

File management provides a hierarchical file system, metadata (size, timestamps, permissions), and operations (create, open, read, write, delete). The OS enforces access control (read/write/execute permissions, ACLs) and maintains consistency (journaling, atomic updates). File caching/buffering improves performance, and directories let users organize files. Example: saving a document uses the OS file API to write bytes to a file; behind the scenes the OS checks your permissions, updates directory entries and inodes, and writes blocks to disk (possibly through a journal to survive crashes).

Device management — interfacing peripherals

The OS mediates access between applications and hardware devices (disks, keyboards, network cards, printers) via device drivers and a unified interface (character/block devices, device files). It handles interrupts, I/O buffering, and policies like exclusive vs. shared access. Modern OSes provide plug-and-play services and abstract device specifics behind driver APIs so programs use standard calls. Example: when you print, the print subsystem sends formatted data to the printer driver, which translates it to the printer's protocol, queues the job, and signals completion via interrupts or callbacks.

Networking

Networking in the OS implements protocol stacks (commonly TCP/IP), sockets or higher-level APIs for applications, routing, and packet buffering. The kernel manages connections, packet I/O, NAT/firewall hooks, and network device drivers. It enforces resource limits (number of sockets, bandwidth) and integrates with IPC and security subsystems. Example: a web server creates a listening TCP socket; when a remote client connects, the OS kernel accepts the connection, queues incoming packets, and delivers data to the server process through the socket API.

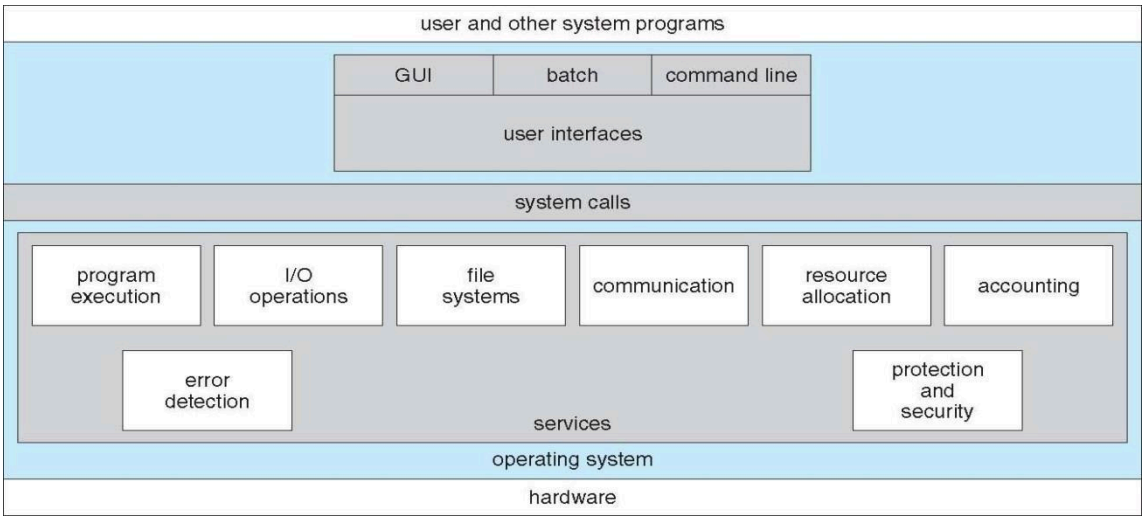
---

5. Operating System Services

Operating-System Services

Q) List and explain the services provided by OS for the user and efficientoperation of system.

An operating system provides an environment for the execution of programs. It provides certainservices to programs and to the users of those programs.



OS provide services for the users of the system, including:

- **User Interfaces** - Means by which users can issue commands to the system. Depending on the operating system these may be a **command-line interface** (e.g. sh, csh, ksh, tcsh, etc.), a **Graphical User Interface** (e.g. Windows, X-Windows, KDE, Gnome, etc.), or a **batch command systems**.
  In Command Line Interface (CLI)- commands are given to the system.

  In Batch interface – commands and directives to control these commands are put in a file andthen the file is executed.

  In GUI systems- windows with pointing device to get inputs and keyboard to enter the text.
- **Program Execution** - The OS must be able to load a program into RAM, run the program, andterminate the program, either normally or abnormally.
- **I/O Operations** - The OS is responsible for transferring data to and from I/O devices, including keyboards, terminals, printers, and files. For specific devices, special functions are provided (device drivers) by OS.
- **File-System Manipulation** – Programs need to read and write files or directories. The servicesrequired to create or delete files, search for a file, list the contents of a file and change the file permissions are provided by OS.

- **Communications** - Inter-process communications, IPC, either between processes running on the same processor, or between processes running on separate processors or separate machines.May be implemented by using the service of OS- like shared memory or message passing.
- **Error Detection** - Both hardware and software errors must be detected and handled appropriately by the OS. Errors may occur in the CPU and memory hardware (such as power failure and memory error), in I/O devices (such as a parity error on tape, a connection failureon a network, or lack of paper in the printer), and in the user program (such as an arithmetic overflow, an attempt to access an illegal memory location).

OS provide services for the efficient operation of the system, including:

- **Resource Allocation** – Resources like CPU cycles, main memory, storage space, and I/O devices must be allocated to multiple users and multiple jobs at the same time.
- **Accounting** – There are services in OS to keep track of system activity and resource usage, either for billing purposes or for statistical record keeping that can be used to optimize future performance.
- **Protection and Security** – The owners of information (file) in multiuser or networked computer system may want to control the use of that information. When several separate processes execute concurrently, one process should not interfere with other or with OS. Protection involves ensuring that all access to system resources is controlled. Security of the system from outsiders must also be done, by means of a password.

User Operating-System Interface

There are several ways for users to interface with the operating system.

i) Command-line interface, or command interpreter, allows users to directly enter commandsto be performed by the operating system.

ii) Graphical user interface (GUI), allows users to interface with the operating system usingpointer device and menu system.

Command Interpreter

- Command Interpreters are used to give commands to the OS. There are multiple command interpreters known as shells. In UNIX and Linux systems, there are several different shells, likethe *Bourne shell, C shell, Bourne-Again shell, Korn shell,* and others.
- The main function of the command interpreter is to get and execute the user-specified command. Many of the commands manipulate files: create, delete, list, print, copy, execute,and so on.

The commands can be implemented in two general ways-

i) The command interpreter itself contains the code to execute the command. For example, a command to delete a file may cause the command interpreter to jump to a particular sectionof its code that sets up the parameters and makes the appropriate system call.

ii) The code to implement the command is in a function in a separate file. The interpreter searches for the file and loads it into the memory and executes it by passing the parameter. Thus by adding new functions new commands can be added easily to the interpreterwithout disturbing it.

Graphical User Interfaces

- A second strategy for interfacing with the operating system is through a userfriendly graphical user interface or GUI. Rather than having users directly enter commands via a command-line interface, a GUI allows provides a mouse-based window-and-menu system as an interface.
- A GUI provides a **desktop** metaphor where the mouse is moved to position its pointer on images, or **icons,** on the screen (the desktop) that represent programs, files, directories, and system functions.
- Depending on the mouse pointer's location, clicking a button on the mouse can invoke a program, select a file or directory—known as a **folder**— or pull down a menu that contains commands.

Services offered to users and system programs:

- Program execution
- I/O operations
- File manipulation
- Communications (IPC)
- Error detection/security

5. Operating System Services

Program execution

The OS provides services to load and run programs: reading an executable from disk, setting up the process address space, linking libraries, and transferring control to the program's entry point. It provides system calls to create new processes, set arguments/environment, and manage process states. Example: launching an application

from a launcher invokes the OS loader to map the binary and shared libraries into memory, initialize stacks and registers, then start execution.

I/O operations

I/O services manage reading from and writing to devices with abstractions (streams, files, sockets) and mechanisms such as buffering, caching, and asynchronous I/O. The OS handles blocking vs nonblocking calls, provides synchronous and asynchronous APIs, and optimizes transfers (e.g., DMA). Example: copying a large file uses buffered I/O; the OS reads blocks into kernel buffers and writes them to the destination, using DMA to move data without CPU cycles for each byte.

File manipulation

Beyond basic storage, the OS offers file manipulation functions: rename, copy, change permissions, truncate, lock/unlock, and directory operations. The OS maintains atomicity where required (rename is typically atomic), manages file descriptors/handles, and provides utilities for searching and metadata updates. Example: when a program renames a file, the OS atomically updates the directory entry so other processes either see the old name or the new name but never a half-state.

Communications (IPC)

Interprocess communication services let processes exchange data or synchronize: pipes, message queues, sockets, shared memory, semaphores, and signals. The OS enforces access control and manages buffers, wakeups, and ordering semantics. Example: a producer process writes log entries to a pipe and a consumer process reads from that pipe; the kernel buffers data and wakes the consumer when new bytes arrive.

Error detection / security

The OS provides mechanisms to detect faults (I/O errors, invalid memory access), report them, and isolate failures (process termination, core dumps). Security services

include authentication, authorization (access control lists, capability checks), auditing, and features like address space layout randomization (ASLR) and SELinux/AppArmor for mandatory access control. Example: if a user tries to read a file they lack permission for, the OS returns an access denied error and logs the attempt if auditing is enabled; if a program dereferences a null pointer, the OS detects the invalid access and terminates that process to protect others.

---

6. User and OS Interface

Interfaces include:

- CLI (text commands)
- GUI (windows, icons)

6. User and OS Interface

User in an Operating System

A *user* is someone who uses the computer system to run programs, access files, and perform tasks. Users do not interact with hardware directly; instead, they interact with the **OS**, which provides commands, applications, and interfaces. The OS ensures that each user gets proper access to resources while maintaining security and fairness.

Role of Users in the OS

1. Interacting with Applications

Users start programs, open files, run commands, and use hardware indirectly.
Example: Double-clicking a browser icon to launch Chrome.

2. Providing Input to the System

The user gives instructions that the OS interprets—typing commands, clicking icons, tapping screens.

Example: Typing mkdir newfolder in Linux to create a directory.

3. Security and Permissions

The OS assigns each user permissions and ownership to protect data.

Example:

One user cannot access another user's private files without permission.

4. Resource Usage

Each user consumes CPU time, memory, disk space, and network access. The OS manages these resources fairly across users.

Example:

Two users logged into a server each run programs; the scheduler divides CPU time.

CLI (Command-Line Interface — text commands)

A CLI is a text-based interface where users type commands and arguments; the shell interprets these, performs parsing/expansion (globbing, variables), and issues system calls on the user's behalf. CLIs are powerful for scripting, automation, and precise control, because commands can be combined with pipes and redirection. Example: in a UNIX shell you might run grep -R "TODO" . | wc -l to recursively search for "TODO" and count matches — the shell creates pipes and processes, the OS schedules them, and the underlying system calls read files and deliver results.

GUI (Graphical User Interface — windows, icons)

A GUI provides graphical elements — windows, menus, icons, buttons — enabling users to interact with the system via mouse, touch, or keyboard in a visually intuitive way. The OS (often with a window manager/compositor) manages drawing, event

delivery (clicks, keystrokes), window placement, and compositing, while toolkits let applications create standard widgets. GUIs are ideal for discoverability and user friendliness. Example: a file manager shows icons for folders and files; when you drag a file into a folder, the GUI sends the corresponding file-operation request to the OS, which moves the file and updates the display to reflect the change.

7. System Calls (with examples)

System calls allow user programs to request OS services and communicate with the kernel.

Major types:

Type	**Windows**	**Linux**
Process Control	CreateProcess()	fork()
File Management	CreateFile()	open()
Device Management	SetConsoleMode()	ioctl()
Info Maintenance	GetCurrentProcessID()	getpid()
Communication	CreatePipe()	pipe()

System Calls

System calls are the **interface between user programs and the kernel**. They provide a way for user level applications to **request services** from the operating system.

For example, when you open a file, your application does not directly interact with the disk. Instead, it makes a **system call** to the kernel, which then performs the necessary operations to access the file.

Common types of system calls include:

- **Process control:** Creating, terminating, and managing processes.
- **File manipulation:** Reading, writing, opening, and closing files.
- **Device manipulation:** Communicating with hardware devices.
- **Information maintenance:** Getting system time, system information, etc.
- **Communication :** Sending and receiving messages between processes.

System calls make it easier for user applications to perform complex operations without directly accessing hardware. This ensures **security**, **stability**, and **hardware independence**.

---

8. System Programs

System programs help develop, execute, and manage programs:

- File management: editors, search tools
- Programming support: compilers, linkers
- Program loading/execution: loaders, debuggers
- Status/info utilities: process statistics, configs
- Communications: network tools

8. System Programs

System programs are software components provided by the operating system or installed alongside it to help users **develop**, **execute**, **manage**, and **monitor** programs. Unlike applications (like browsers or games), system programs directly support the functioning of the OS and the execution of user programs. They act as essential tools for developers, administrators, and even regular users.

A. File Management (editors, search tools)

File management system programs allow users to **create, modify, organize, and locate files** on the computer. Text editors (like Notepad, nano, or Vim) let users create or edit source code and configuration files. File search tools (like Windows Search or Linux's find and grep) help locate files by name or content. These programs interact directly with the OS file system, making file manipulation easier.

Example:

A developer uses grep "main" to search for all files containing the word "main" across their project directory.

B.Programming Support (compilers, linkers)

Programming support tools assist in writing and building software. **Compilers** translate high-level languages (like C, Java, or Python) into machine code that the CPU can execute. **Linkers** combine compiled modules and libraries into a complete executable program. These tools rely on the OS for memory, file access, and process management while creating output files.

Example:

A C program (program.c) is compiled with gcc program.c -o program. The compiler translates the code, and the linker connects it with system libraries to create the executable file.

C. Program Loading and Execution (loaders, debuggers)

Loaders and debuggers support running and analyzing programs. A **loader** is responsible for loading an executable program into memory, setting up the address space, and starting execution. **Debuggers** (like GDB or Visual Studio Debugger) allow programmers to run a program step-by-step, inspect variables, and detect bugs. These tools interact heavily with OS mechanisms such as process creation, memory allocation, and exception handling.

Example:

A developer runs a program under a debugger: gdb ./a.out. They set breakpoints, examine memory, and trace execution to fix errors.

D. Status and Information Utilities (process statistics, configs)

These are programs that provide **system information** such as CPU usage, memory consumption, running processes, device status, and configuration details. Tools like Task Manager (Windows), top or htop (Linux), and System Information utilities give real-time insights that help optimize performance or troubleshoot issues.

Example:

Using top in Linux shows the list of active processes, their CPU usage, memory use, and process IDs, helping administrators identify resource-heavy applications.

E. Communications (network tools)

Communication system programs handle data exchange between systems through networks. These tools help test connectivity, transfer files, configure network settings, and diagnose communication problems. Examples include ping (test connection), ssh (secure remote login), ftp/sftp (file transfer), and netstat (view network connections). The OS networking subsystem works with these tools to send and receive packets, open ports, and manage protocols.

Example:

An administrator uses ping google.com to check if the computer can reach an external network, verifying internet connectivity

MODULE–2: PROCESS MANAGEMENT

What is a Process?

A process is a program in execution.

A program is a collection of instructions which are useful to perform some operation/task.
When a program is compiled, it resides on the hard disk (secondary memory).
At the time of execution, the Operating System loads the program from secondary memory into main memory, and it becomes a process.

Example:
A C program during compilation stays on hard disk.
During execution, OS loads it into main memory.

Process Representation in Main Memory (Image / Address Space)

Every process is represented internally in main memory using the following segments:

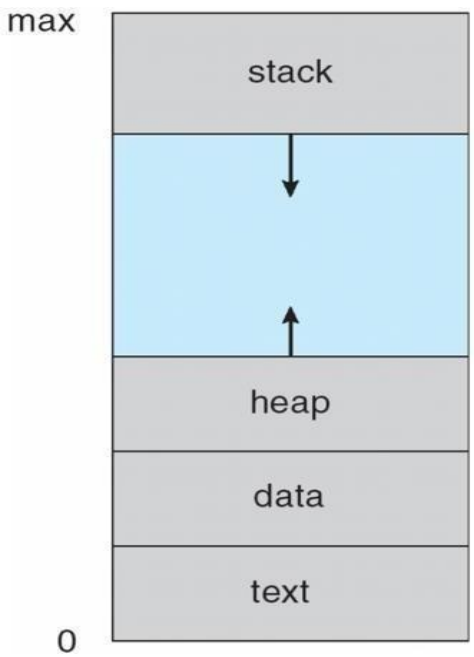


Figure: Process in memory. ***Process***

Process Concept

- A process is a program under execution.
- Its current activity is indicated by PC (Program Counter) and the contents of the processor'sregisters.

***The Process:* Process memory is divided into four sections as shown in the figure below:**

- The **stack** is used to store temporary data such as local variables, function parameters, functionreturn values, return address etc.
- The **heap** which is memory that is dynamically allocated during process run time ☛ The **data** section stores global variables.
- The **text** section comprises the compiled program code.
- Note that, there is a free space between the stack and the heap. When the stack is full, it growsdownwards and when the heap is full, it grows upwards.

---

1) Text Segment
- Also called code segment
- Stores program instructions
- Every program contains several instructions

---

2) Data Segment
- Stores global variables
- Stores static variables

---

3) Heap
- Memory allocated during runtime
- Used for dynamic memory allocation
- Example: pointers using
  malloc(), calloc(), realloc()

---

4) Stack
- Used to store:
  - Function parameters
  - Local variables
  - Return values

Process Concept

Q) Illustrate with a neat sketch, the process states and process control block.

Process State

A Process has 5 states. Each process may be in one of the following states –

1. **New** - The process is in the stage of being created.
2. **Ready** - The process has all the resources it needs to run. It is waiting to be assigned tothe processor.
3. **Running** – Instructions are being executed.
4. **Waiting** - The process is waiting for some event to occur. For example, the process may be waiting for keyboard input, disk access request, inter-process messages, a timer to go off, or a child process to finish.
5. **Terminated** - The process has completed its execution.

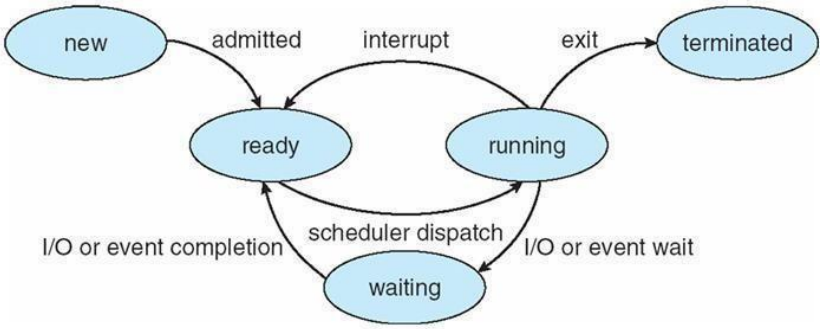


Figure: Diagram of process state

Process Control Block

For each process there is a Process Control Block (PCB), which stores the process-specificinformation as shown below –

- **Process State** – The state of the process may be new, ready, running, waiting, and so on.
- **Program counter** – The counter indicates the address of the next instruction to be executed forthis process.
- **CPU registers** - The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general- purpose registers. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.
- **CPU scheduling information**- This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
- **Memory-management information** – This includes information such as the value of the base and limit registers, the page tables, or the segment tables.
- **Accounting information** – This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- **I/O status information** – This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

• Process Control Block (PCB)

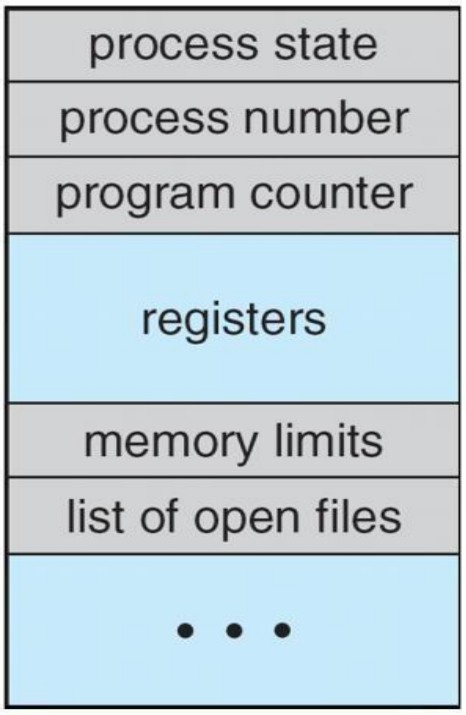
Process Control Block

For each process there is a Process Control Block (PCB), which stores the process-specificinformation as shown below –

- **Process State** – The state of the process may be new, ready, running, waiting, and so on.

- **Program counter** – The counter indicates the address of the next instruction to be executed forthis process.
- **CPU registers** - The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general- purpose registers. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.
- **CPU scheduling information**- This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
- **Memory-management information** – This includes information such as the value of the base and limit registers, the page tables, or the segment tables.
- **Accounting information** – This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- **I/O status information** – This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

The PCB simply serves as the repository for any information that may vary from process to



process.

Figure: Process control block (PCB)

2. Process Scheduling

Scheduling Queue:

1. **Job Queue**
2. **Ready Queue**
3. **Device Queue**

Diagram Representation of Process Scheduling

1. Job Queue

It contains the list of all the processes which are in the system. Job queue will reside in secondary memory (such as hard disk).

2. Ready Queue

It contains the list of processes which are ready for execution by the CPU. Ready queue will reside in the main memory.

3. Device Queue

It contains the list of processes which are waiting for I/O operation.

SCHEDULING ALGORITHMS

🎬 CPU scheduling deals with the problem of deciding which of the processes inthe ready-queue is to be allocated the CPU.

Following are some scheduling algorithms:

1. FCFS scheduling (First Come First Served)
2. Round Robin scheduling
3. SJF scheduling (Shortest Job First)
4. SRT scheduling
5. Priority scheduling

6. Multilevel Queue scheduling
7. Multilevel Feedback Queue scheduling

FCFS Scheduling

- The process that requests the CPU first is allocated the CPU first.
- The implementation is easily done using a FIFO queue. Procedure:
  1. When a process enters the ready-queue, its PCB is linked onto the tail ofthe queue.
  2. When the CPU is free, the CPU is allocated to the process at the queue's head.
  3. The running process is then removed from the queue.
- Advantage:
  1. Code is simple to write & understand.
- Disadvantages:
  1. **Convoy effect:** All other processes wait for one big process to get off the CPU.
  2. Non-preemptive (a process keeps the CPU until it releases it).
  3. Not good for time-sharing systems.
  4. The average waiting time is generally not minimal.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Example: Suppose that the processes arrive in the order P1, P2,P3.
- The Gantt Chart for the schedule is as follows:

- Waiting time for $P1$ = 0; $P2$ = 24; $P3$ =27

  Average waiting time: (0 + 24 + 27)/3 = 17ms
- Suppose that the processes arrive in the order P2, P3,P1.
- The Gantt chart for the schedule is as follows:

  Waiting time for P1 = 6;P2 = 0; P3 =3
- Average waiting time: (6 + 0 + 3)/3 = 3ms

SJF Scheduling

- The CPU is assigned to the process that has the smallest next CPU burst.
- If two processes have the same length CPU burst, FCFS scheduling is used to breakthe tie.
- For long-term scheduling in a batch system, we can use the process time limitspecified by the user, as the 'length'
- SJF can't be implemented at the level of short-term scheduling, because there isno way to know the length of the next CPU burst ➡ Advantage:

  1. The SJF is optimal, i.e. it gives the minimum average waiting time for a given set of processes.

- Disadvantage:

  1. Determining the length of the next CPU burst.

- SJF algorithm may be either 1) non-preemptive or 2) preemptive.

1. Non preemptive SJF

The current process is allowed to finish its CPU burst.

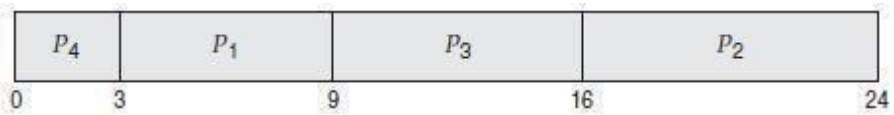
2. Preemptive SJF

If the new process has a shorter next CPU burst than what is left of the executing process, that process is preempted. It is also known as **SRTF** scheduling (**Shortest-Remaining-Time-First**).

- Example (for non-preemptive SJF): Consider the following set of processes,with the length of the CPU-burst time given in milliseconds.

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

- For non-preemptive SJF, the Gantt Chart is as follows:



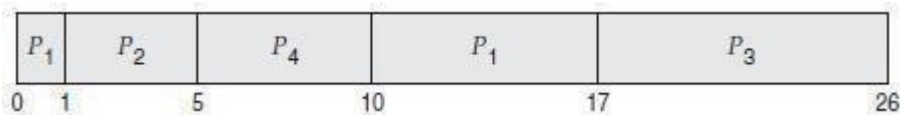
- Waiting time for P1 = 3; P2 = 16; P3 = 9; 4=0 ➡ Average waiting time: (3 + 16 + 9 +0)/4= 7

preemptive SJF/SRTF: Consider the following set of processes, with the length

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

of the CPU- burst time given in milliseconds.

- For preemptive SJF, the Gantt Chart is as follows:



- The average waiting time is ((10 - 1) + (1 - 1) + (17 - 2) + (5 - 3))/4 = 26/4 =6.5.

Priority Scheduling

- A priority is associated with each process.
- The CPU is allocated to the process with the highest priority.
- Equal-priority processes are scheduled in FCFS order. Priorities can be defined either internally or externally.
    1. ***Internally-defined*** priorities.
        - Use some measurable quantity to compute the priority of a process.
        - For example:

          time limits, memory requirements, no. f open files.
    2. ***Externally-defined*** priorities.
        - Set by criteria that are external to the OS Forexample:
        - importance of the process, political factors
- Priority scheduling can be either preemptive or non- preemptive.1.***Preemptive***

  The CPU is preempted if the priority of the newly arrived process ishigher than the priority of the currently running process.

  2. *Non Preemptive*

The new process is put at the head of the ready-queue

- Advantage: Higher priority processes can be executed first.
- Disadvantage:
- A major problem with priority scheduling algorithms is **indefinite blocking, or starvation**. A process that is ready to run but waiting for the CPU can be considered blocked. A priority scheduling algorithm

can leave some low- priority processes waiting indefinitely. In a heavily loaded computer system, a steady stream of higher-priority processes can prevent a low-priority process from ever getting the CPU.

- Solution: ***Aging*** is a technique of increasing priority of processes that wait in system for a long time.
- Example: Consider the following set of processes, assumed to have arrived at time 0, in the order PI, P2, ..., P5, with the length of the CPU-burst time given

in milliseconds.

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- The Gantt chart for the schedule is as follows:

- The average waiting time is 8.2milliseconds.

Round Robin Scheduling

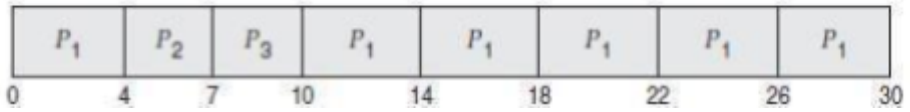
- Designed especially for time sharing systems.
- It is similar to FCFS scheduling, but with preemption.
- A small unit of time is called a ***time quantum****(or time slice).*
- Time quantum is ranges from 10 to 100ms.
- The ready-queue is treated as a **circular queue**.
- The CPUscheduler
- goes around the ready-queue and
- allocates the CPU to each process for a time interval of up to 1 time quantum.
- To implement:

  The ready-queue is kept as a FIFO queue of processes
- CPUscheduler
  1. Picks the first process from the ready-queue.
  2. Sets a timer to interrupt after 1 time quantum and
  3. Dispatches the process.
- One of two things will then happen.
  1. The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily.
  2. If the CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the OS. The process will be put at the tail of the ready-queue.
- Advantage:
- Higher average turnaround than SJF.

- Disadvantage:
- Better response time than SJF.
- Example: Consider the following set of processes that arrive at time 0, with thelength of the CPU-burst time given in milliseconds.

Process	Burst Time
P_1	24
P_2	3
P_3	3

The Gantt chart for the schedule is as follows:



- The average waiting time is 17/3 = 5.66milliseconds.
- *The RR scheduling algorithm is preemptive.*

  No process is allocated the CPU for more than *1* time quantum in a row.If a process' CPU burst exceeds *1* time quantum, that process is preempted and is put back in the ready-queue.
- The performance of algorithm depends heavily on the size of the time quantum.
  1. If time quantum=very large, RR policy is the same as the FCFS policy.
  2. If time quantum=very small, RR approach appears to the users as though each of n processes has its own processor running at l/n the speed of the real processor.
- In software, we need to consider the effect of context switching on the performance of RR scheduling
  1. Larger the time quantum for a specific process time, less time is spend on context switching.
  2. The smaller the time quantum, more overhead is added for the purpose of context- switching.

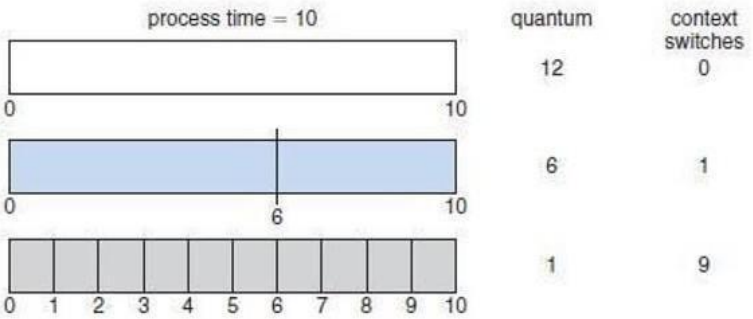
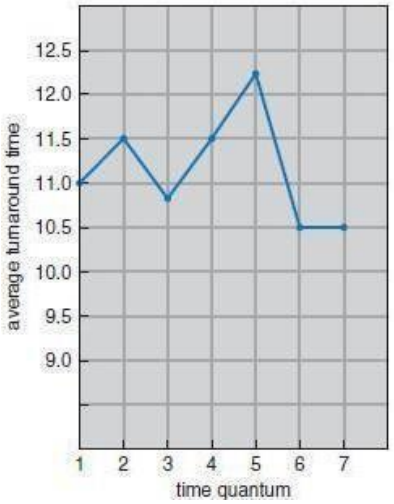


Fig: How a smaller time quantum increases context switches



process	time
P_1	6
P_2	3
P_3	1
P_4	7

Fig: How turnaround time varies with the time quantum

PROCESS SCHEDULING

Basic Concepts

- In a single-processor system,
  - Only one process may run at a time.
  - Other processes must wait until the CPU is rescheduled.
- Objective of multiprogramming:
  - To have some process running at all times, in order to maximize CPUutilization.

CPU-I/0 Burst Cycle

- Process execution consists of a cycle of
  - CPU execution and
  - I/O wait
- Process execution begins with a CPU burst, followed by an I/O burst, then another CPU burst, etc…
- Finally, a CPU burst ends with a request to terminate execution.
- An I/O-bound program typically has many short CPU bursts.
- A CPU-bound program might have a few long CPU bursts.

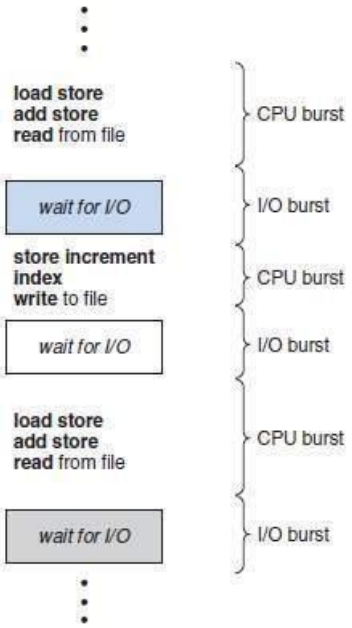


Fig Alternating sequence of CPU and I/O bursts

✔ The duration of CPU bursts vary from process to process and from computer to computer.

✔ The **frequency curve** is as shown below **figure.**

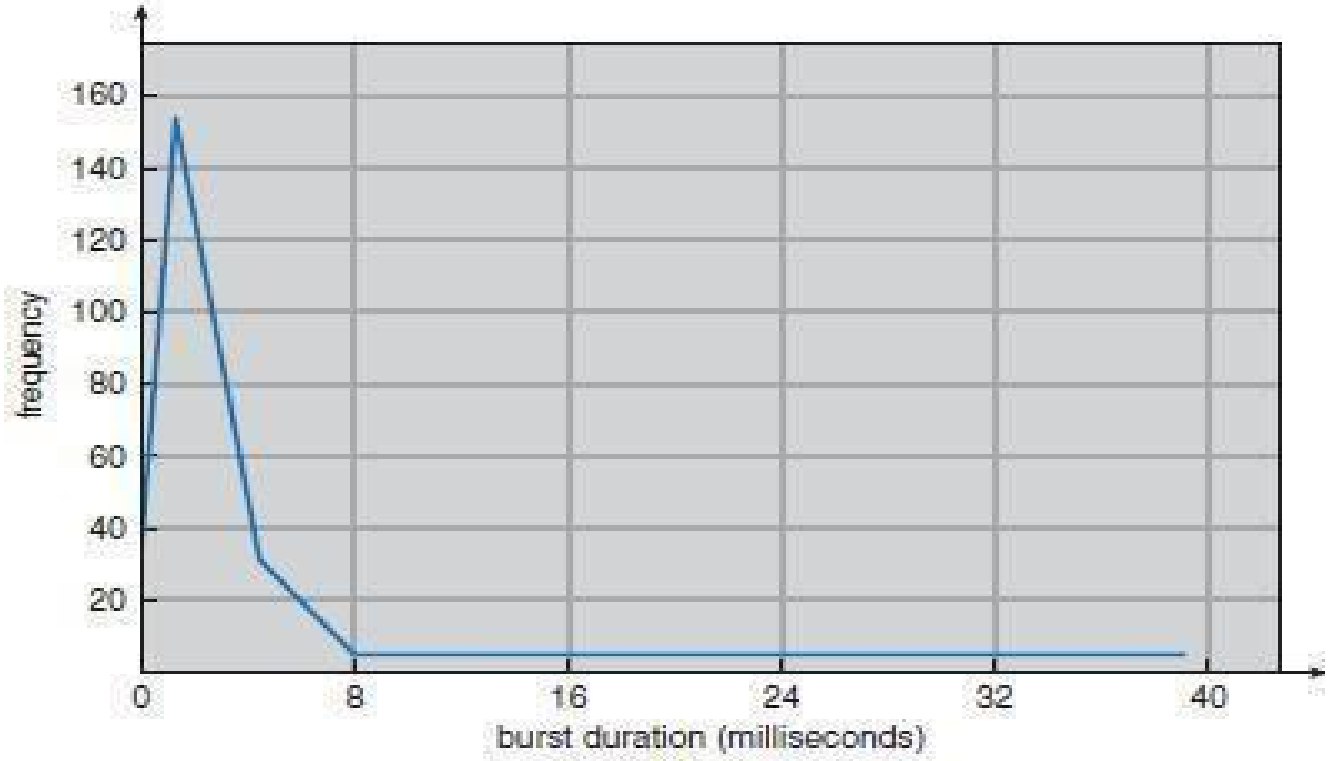


Fig: Histogram of CPU-burst durations

CPU Scheduler

- This scheduler
  - selects a waiting-process from the ready-queue and
  - allocates CPU to the waiting-process.
- The ready-queue could be a FIFO, priority queue, tree and list.
- The records in the queues are generally process control blocks (PCBs) of the processes. **CPU Scheduling**
- Four situations under which CPU scheduling decisions take place:
  1. When a process switches from the running state to the waiting state. For ex; I/Orequest.
  2. When a process switches from the running state to the ready state. For ex:when an interrupt occurs.
  3. When a process switches from the waiting state to the ready state. For ex:completion of I/O.
  4. When a process terminates.
- Scheduling under 1 and 4 is non- preemptive. Scheduling under 2 and 3 is preemptive.

Non Preemptive Scheduling

- Once the CPU has been allocated to a process, the process keeps the CPU until itreleases the CPU either
  - by terminating or
  - by switching to the waiting state. **Preemptive Scheduling**
- This is driven by the idea of prioritized computation. Processes that are runnable may be temporarily suspended
- Disadvantages:
  1. Incurs a cost associated with access to shared-data.
  2. Affects the design of the OS kernel.

Dispatcher

It gives control of the CPU to the process selected by the short-term scheduler.

- The function involves:
  1.Switching context
  2.Switching to user mode&
  3. Jumping to the proper location in the user program to restart that program
- It should be as fast as possible, since it is invoked during every process switch.
- ***Dispatch latency*** means the time taken by the dispatcher to
  - stop one process and
  - start another running.

SCHEDULING CRITERIA:

In choosing which algorithm to use in a particular situation, depends upon the properties of the various algorithms. Many criteria have been suggested for comparing CPU- scheduling algorithms. The criteria include the following:

1. **CPU utilization:** We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system).
2. **Throughput:** If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput. For long processes, this rate may be one process per hour; for short transactions, it may be ten processes per second.
3. **Turnaround time.** This is the important criterion which tells how long it takes to execute that process. The interval from the time of submission of a process to the time of completion is the turnaround time. Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/0.
4. **Waiting time:** The CPU-scheduling algorithm does not affect the amount of time during which a process executes or does I/0, it affects only the amount of time thata process spends waiting in the ready queue. Waiting time is the sum of the periods spent waiting in the ready queue.
5. **Response time:** In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is the time from the submission of a request until the first responseis produced. This measure, called response time, is the time it takes to start responding, not the time it takes to output the response. The turnaround time is generally limited by the speed of the output device.

Flow of Process Scheduling

- **Ready Queue → CPU**
- From **CPU**:

- If **I/O request** occurs → Process moves to **I/O Queue** → **I/O** → back to **Ready Queue**
- If **Time slice expired** → Process moves back to **Ready Queue**
- If **Fork a child** → **Child executes**
- If **Waiting for an event** → Process waits until **Interrupt occurs** → back to **Ready Queue**

Scheduling Queues

- As processes enter the system, they are put into a job queue, which consists of all processes inthe system.
- The processes that are residing in main memory and are ready and waiting to execute are kepton a list called the ready queue. This queue is generally stored as a linked list.
- A ready-queue header contains pointers to the first and final PCBs in the list. Each PCBincludes a pointer field that points to the next PCB in the ready queue.

Ready Queue and Various I/O Device Queues

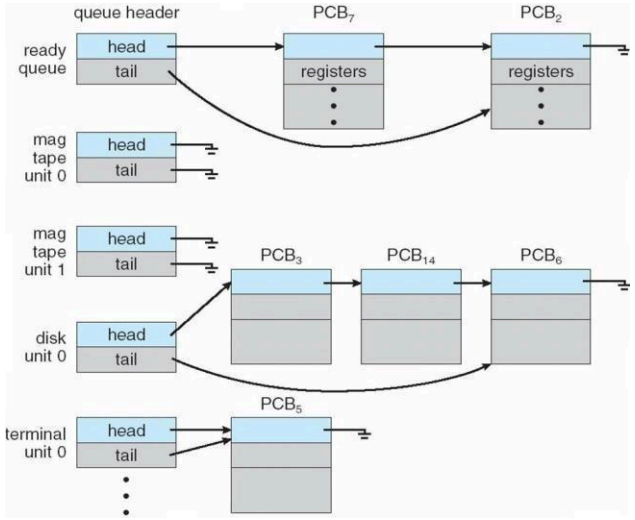


Figure: The ready queue and various I/O device queues

A common representation of process scheduling is a ***queueing diagram***. Each rectangular box in the diagram represents a queue. Two types of queues are present: the ready queue and a setof device queues. The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.

- A new process is initially put in the ready queue. It waits in the ready queue until it is selected for execution and is given the CPU. Once the process is allocated the CPU and is executing, one of several events could occur:
- The process could issue an I/O request, and then be placed in an I/O queue.
- The process could create a new subprocess and wait for its termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt,and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state, and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues.

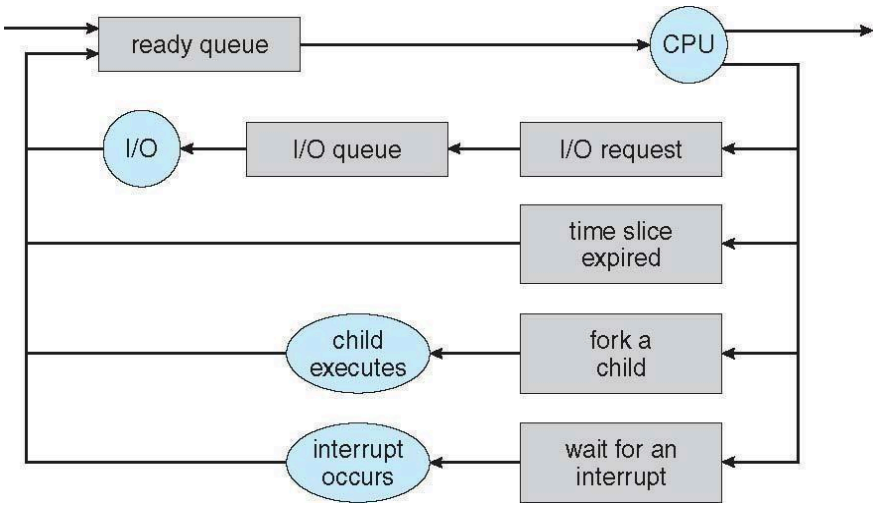


Figure: Queueing-diagram representation of process scheduling.

Schedulers

Schedulers are software which selects an available program to be assigned to CPU.

- A **long-term scheduler or Job scheduler** – selects jobs from the job pool (of secondary memory, disk) and loads them into the memory.

  If more processes are submitted, than that can be executed immediately, such processes will be in secondary memory. It runs infrequently, and can take time to select the next process.

- **The short-term scheduler, or CPU Scheduler** – selects job from memory and assigns theCPU to it. It must select the new process for CPU frequently.
- **The medium-term scheduler** - selects the process in ready queue and reintroduced into the memory.

Processes can be described as either:

- I/O-bound process – spends more time doing I/O than computations, CPU-bound process – spends more time doing computations and few I/O operations.

An efficient scheduling system will select a good mix of **CPU-bound** processes and **I/O bound** processes.

- If the scheduler selects **more I/O bound process**, then I/O queue will be full and readyqueue will be empty.
- If the scheduler selects **more CPU bound process**, then ready queue will be full and I/Oqueue will be empty.

Time sharing systems employ a **medium-term scheduler**. It swaps out the process from ready queue and swap in the process to ready queue. When system loads get high, this scheduler will swap one or more processes out of the ready queue for a few seconds, in order to allow smaller faster jobs to finish up quickly and clear the system.

Advantages of medium-term scheduler –

- To remove process from memory and thus reduce the degree of multiprogramming(number of processes in memory).
- To make a proper mix of processes (CPU bound and I/O bound)

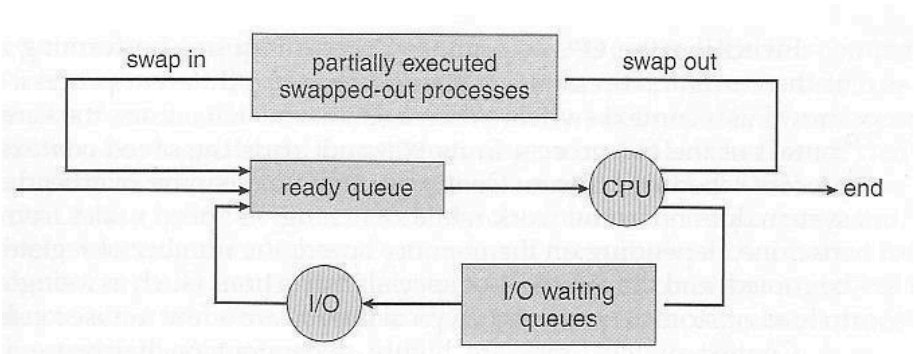


Figure 3.8 Addition of medium-term scheduling to the queueing diagram.

Operations on Processes

Q) Demonstrate the operations of process creation and process termination in UNIXProcess Creation

- A process may create several new processes. The creating process is called a **parent** process, and the new processes are called the **children** of that process. Each of these newprocesses may in turn create other processes. Every process has a unique process ID.
- On typical Solaris systems, the process at the top of the tree is the '**sched**' process with PID of 0. The '**sched**' process creates several children processes – **init**, **pageout** and **fsflush**. Pageout and fsflush are responsible for managing memory and file systems. The init process with a PID of 1, serves as a parent process for all user processes.

A process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its

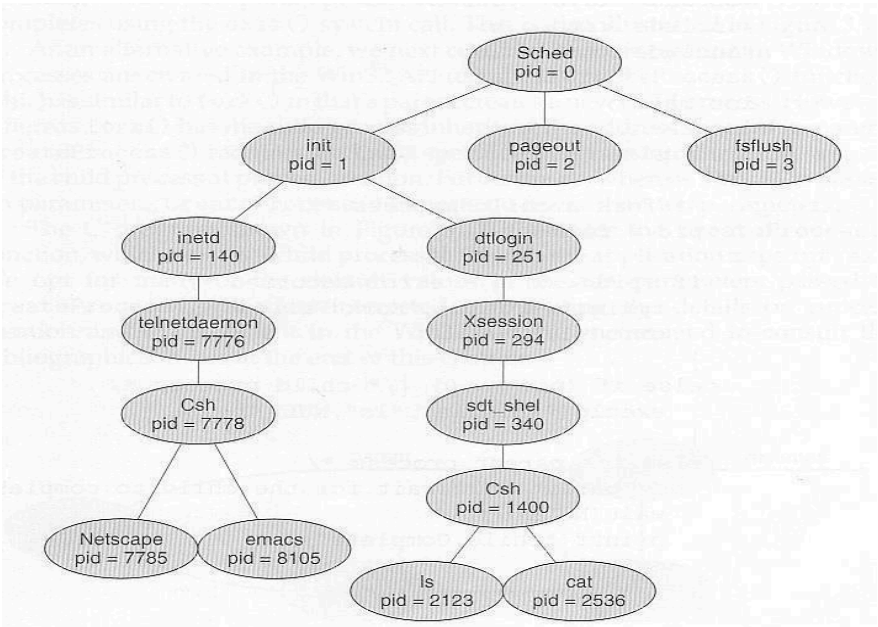


Figure 3.9 A tree of processes on a typical Solaris system.

task. When a process creates a subprocess, the subprocess may be able to obtain its resources in two ways:

- directly from the operating system
- Subprocess may take the resources of the parent process. The resource can be taken from parent in two ways –
  - The parent may have to partition its resources among its children ▪ Share the resources among several children.

There are two options for the parent process after creating the child:

- Wait for the child process to terminate and then continue execution. The parent makes a wait()system call.
- Run concurrently with the child, continuing to execute without waiting. Two possibilities for the address space of the child relative to the parent:

- The child may be an exact duplicate of the parent, sharing the same program and data segments in memory. Each will have their own PCB, including program counter, registers, and PID. This is the behaviour of the **fork** system call in UNIX.
- The child process may have a new program loaded into its address space, with all new code and data segments. This is the behaviour of the **spawn** system calls in Windows.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) {/* error occurred */
      fprintf(stderr, "Fork Failed");
      exit (-1) ;
    }
    else if (pid == 0} {/* child process */
      execlp("/bin/ls","ls",NULL);
    }
    else {/* parent process */
      /* parent will wait for the child to complete */
      wait(NULL);
      printf("Child Complete");
      exit (0) ;
    }
}
```

Figure 3.10 C program forking a separate process.

In UNIX OS, a child process can be created by **fork()** system call. The **fork** system call, if successful, returns the PID of the child process to its parents and returns a zero to the child process. If failure, it returns -1 to the parent. Process IDs of current process or its direct parent can be accessed using the getpid( ) and getppid( ) system calls respectively.

The parent waits for the child process to complete with the wait() system call. When the child process completes, the parent process resumes and completes its execution.

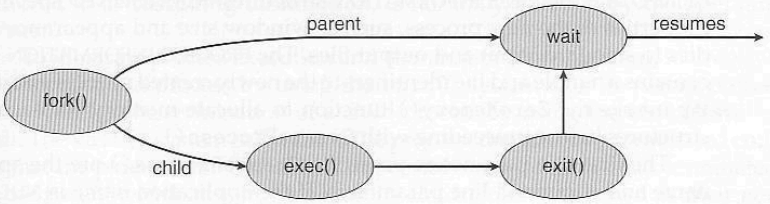


Figure 3.11 Process creation.

In windows the child process is created using the function **createprocess( )**. The createprocess( ) returns 1, if the child is created and returns 0, if the child is not created.

Process Termination

- A process terminates when it finishes executing its last statement and asks the operating systemto delete it, by using the **exit** () system call. All of the resources assigned to the process like memory, open files, and I/O buffers, are deallocated by the operating system.
- A process can cause the termination of another process by using appropriate system call. The parent process can terminate its child processes by knowing of the PID of the child.
- A parent may terminate the execution of children for a variety of reasons, such as:
- The child has exceeded its usage of the resources, it has been allocated.
- The task assigned to the child is no longer required.
- The parent is exiting, and the operating system terminates all the children. This is called **cascading termination**.

Operations on Processes

1. Process Creation

A process is created when the OS needs to execute a program.

Reasons for Process Creation

- System initialization (e.g., startup services)
- A running process creates a child process
- User request (running an application)
- job execution

Steps in Process Creation

1. Assign Process ID (PID)
   The OS assigns a unique identifier to the process.
2. Allocate Resources
   - CPU time
   - Memory

- I/O devices
- Files

3. Create Process Control Block (PCB)
   The PCB stores:
   - Process state
   - Program counter
   - CPU registers
   - Memory limits
   - Open files
   - Scheduling information
4. Load Program into Memory
   The executable code is loaded into main memory.
5. Initialize Process State
   Initial state is usually New, then moved to Ready.

Parent–Child Relationship

- The creating process is called the parent
- The created process is the child
- A child may:
  - Share resources with the parent
  - Have its own independent resources

Examples

- UNIX/Linux: fork(), exec()
- Windows: CreateProcess()

---

2. Process Termination

A process terminates when it finishes execution or is forced to stop.

Reasons for Process Termination

- Normal completion of program
- Error or exception (e.g., divide by zero)
- User or system request
- Parent process termination

Steps in Process Termination

1. Return Status to Parent
   The process provides an exit status.
2. Release Resources
   - Memory

- I/O devices
- Files
- CPU allocation

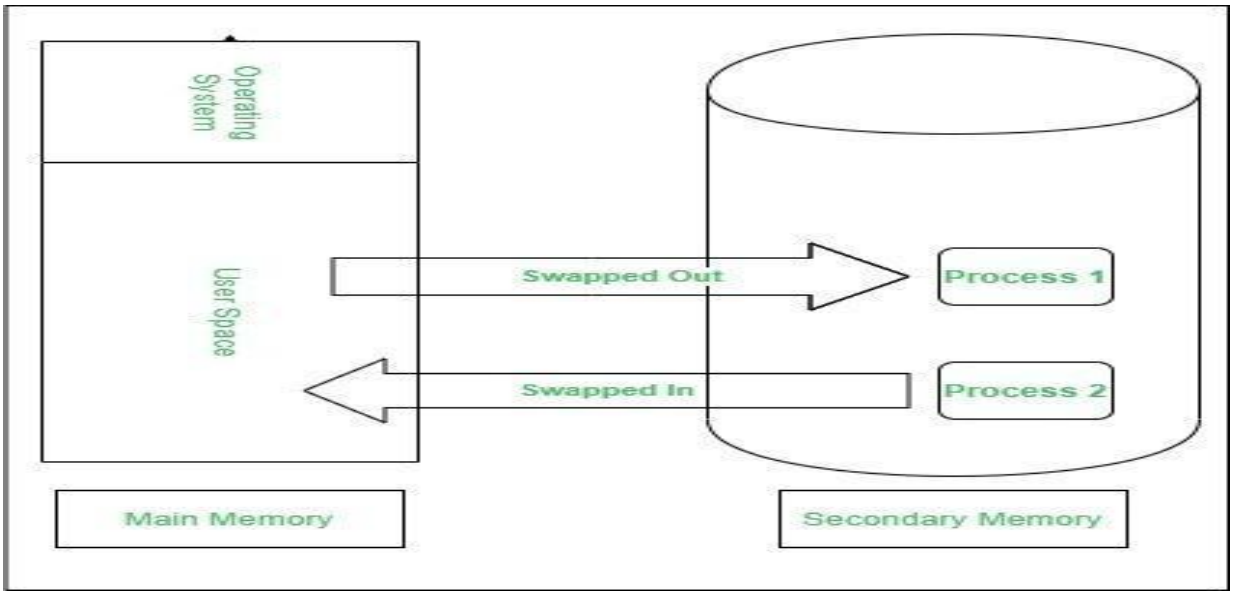
3. Delete PCB
   The OS removes the process control block.
4. Change Process State
   The process moves to the Terminated state.

Zombie and Orphan Processes

- Zombie Process:
  - Process has finished execution but PCB still exists
  - Parent hasn't read exit status
- Orphan Process:
  - Parent terminates before child
  - Child is adopted by init (Linux)

• Schedulers

- **Long-term** scheduler (job scheduler)
- **Short-term** scheduler (CPU scheduler)
- **Medium-term** scheduler (swapping)



. Context Switching

Context switching is the process where the CPU stops running one process, saves its current state, and loads the saved state of another process so that multiple processes can share the CPU effectively.

Need in Multitasking

It is essential in multitasking systems where many processes need CPU time.

- In multitasking, the CPU keeps switching between processes.
- This makes it seem like processes are running at the same time, even though the CPU works on one process at a time.
- Without context switching, one process could monopolize the CPU, and others would have to wait indefinitely

Role in Scheduling

- The scheduler decides which process should run next based on a scheduling algorithm (like Round Robin, Priority Scheduling, etc.).
- Context switching executes that decision by stopping the current process and starting the chosen one.
- The dispatcher is the component that actually performs the switch.

Context Switching Triggers

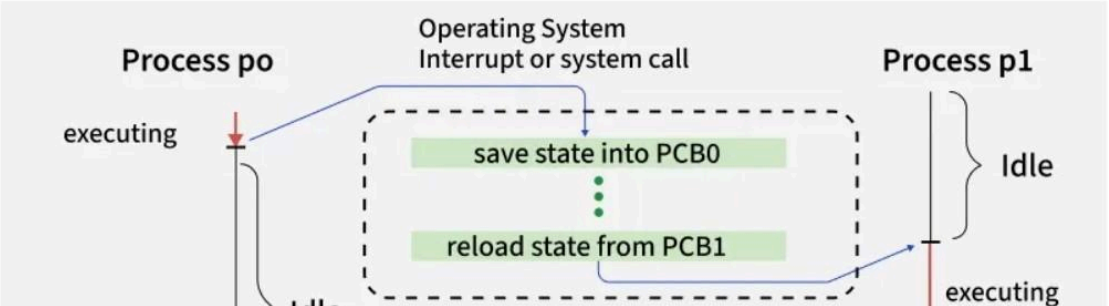
Context Switching Happen:

- When a high-priority process comes to a ready state (i.e. with higher priority than the running process).
- An Interrupt occurs.
- User and kernel-mode switch (It is not necessary though)
- Pre-emptive CPU scheduling is used.

When switching between user mode and kernel/user mode is necessary, operating systems use the kernel/user switch.

Working Process of Context Switching

Let us take examples of two processes **po** and **p1** that execute in interleaved manner



State Diagram of Context Switching

Explanation of the Above Diagram

The diagram illustrates how the CPU alternates execution between two processes — p0 and p1 — through context switching.

1. Execution Phase (Process p0)

- Process p0 is running while p1 remains idle.
- At some point, an interrupt or system call occurs (e.g., a timer interrupt or I/O completion).

2. Saving Current State (p0 → PCB0)

- The operating system pauses p0.
- The current state of p0 (register values, program counter, etc.) is saved into its Process Control Block (PCB0).
- This ensures that p0 can later resume exactly where it left off.

3. Loading New Process State (PCB1 → p1)

- The OS retrieves the saved state of p1 from PCB1.
- This step restores p1's CPU context so it can continue execution from its last saved point.

4. Execution Phase (Process p1)

- Now, p1 is running while p0 is idle.
- Another interrupt or system call occurs, triggering another switch.

5. Saving State of p1 (p1 → PCB1): The OS saves the current state of p1 into PCB1.

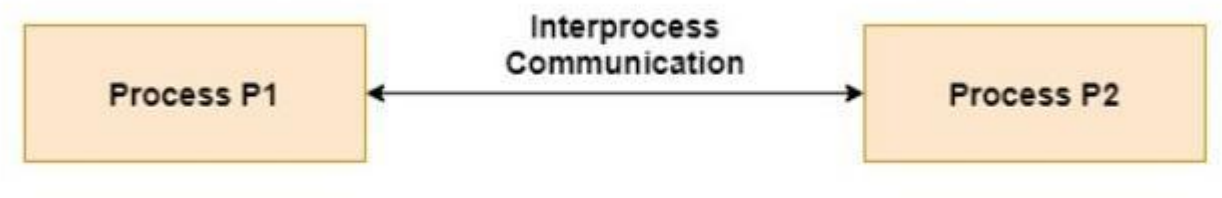
6. Reloading State of p0 (PCB0 → p0)

- The saved state of p0 from PCB0 is loaded back into the CPU registers.
- p0 resumes execution right where it left off.

---

4 Inter-process Communication (IPC)

Interprocess Communication



Q) What is interprocess communication? Explain types of IPC.

Interprocess Communication- Processes executing may be either co-operative or independent processes.

- ***Independent Processes*** – processes that cannot affect other processes or be affected by other processes executing in the system.
- ***Cooperating Processes*** – processes that can affect other processes or be affected by other processes executing in the system.

Co-operation among processes are allowed for following reasons –

- **Information Sharing** - There may be several processes which need to access the same file. Sothe information must be accessible at the same time to all users.
- **Computation speedup** - Often a solution to a problem can be solved faster if the problem canbe broken down into sub-tasks, which are solved simultaneously (particularly when multiple processors are involved.)
- **Modularity** - A system can be divided into cooperating modules and executed by sending information among one another.
- **Convenience** - Even a single user can work on multiple tasks by information sharing.

Cooperating processes require some type of inter-process communication. This is allowed by two models:

1. Shared Memory systems 2.
Message passing systems.

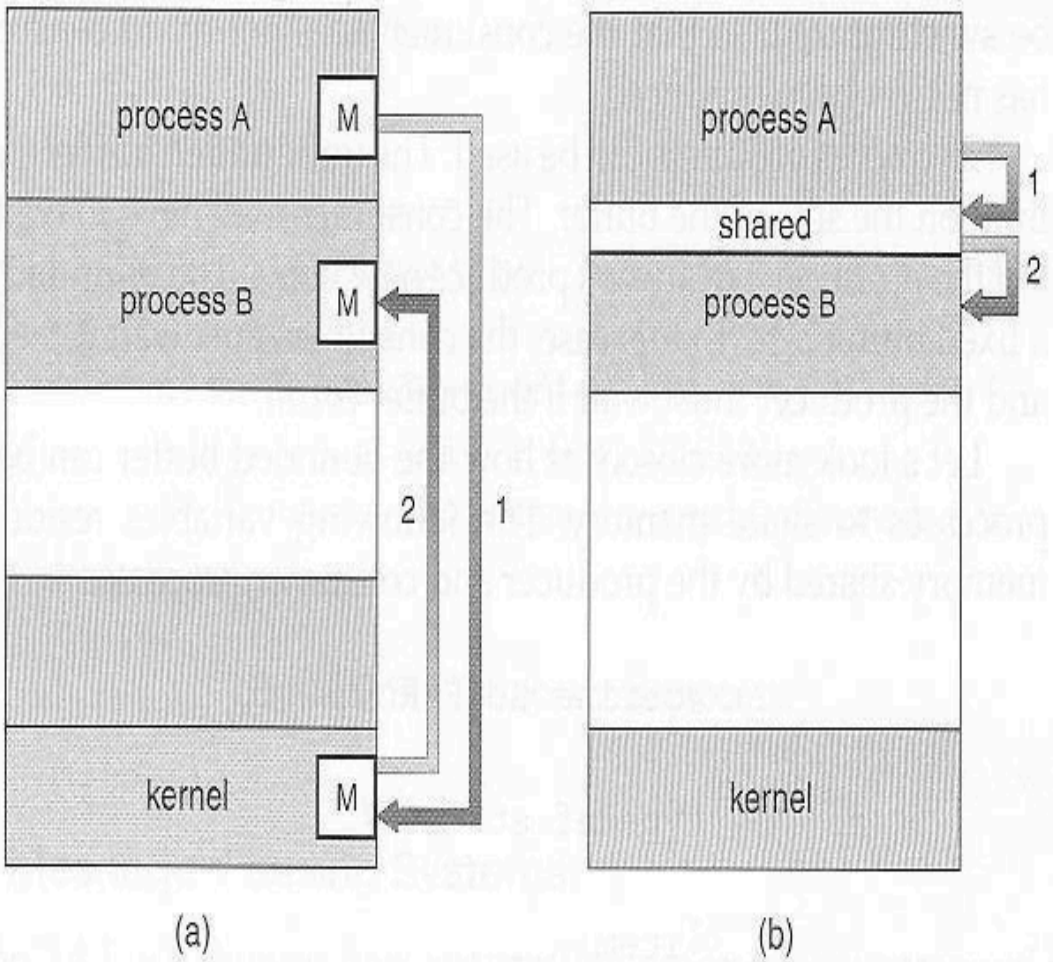


Figure 3.13 Communications models. (a) Message passing. (b) Shared memory.

Sl NO		**Shared Memory**	**Message passing**
	1.	A region of memory is shared by communicating processes, into which the information is written and read	Message exchange is done among the processes by using objects.
	2.	Useful for sending large block of data	Useful for sending small data.
	3.	System call is used only to create shared memory	System call is used during every read and write operation.
	4.	Message is sent faster, as there are no system calls	Message is communicated slowly.

- **Shared Memory** is faster once it is set up, because no system calls are required and access occurs at normal memory speeds. Shared memory is generally preferable when large amounts of information must be shared quickly on the same computer.
- **Message Passing** requires system calls for every message transfer, and is therefore slower, but it is simpler to set up and works well across multiple computers. Message passing is generally preferable when the amount and/or frequency of data transfers is small.

Shared-Memory Systems

- A region of shared-memory is created within the address space of a process, which needs to communicate. Other process that needs to communicate uses this shared memory.
- The form of data and position of creating shared memory area is decided by the process. Generally, a few messages must be passed back and forth between the cooperating processes first in order to set up and coordinate the shared memory access.
- The process should take care that the two processes will not write the data to the shared memory at the same time.

Message-Passing Systems

A mechanism to allow process communication without sharing address space. It is used in distributed systems.

- Message passing systems uses system calls for "send message" and "receive message".
- A communication link must be established between the cooperating processes before messagescan be sent.
- There are three methods of creating the link between the sender and the receiver- ○ Direct or indirect communication (naming) ○ Synchronous or asynchronous communication (Synchronization) ○ Automatic or explicit buffering.

1. Naming

Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication.

a) Direct communication the sender and receiver must explicitly know each other's name. The syntaxfor send() and receive() functions are as follows-

- **send** (*P, message*) – send a message to process P ■ **receive**(*Q, message*) – receive a message from process Q

Properties of communication link:

- A link is established automatically between every pair of processes that wants tocommunicate. The processes need to know only each other's identity to communicate.
- A link is associated with exactly one pair of communicating processes Between each pair, there exists exactly one link.

Types of addressing in direct communication –

- Symmetric addressing – the above-described communication is symmetric communication.Here both the sender and the receiver processes have to name each other to communicate.
- Asymmetric addressing – Here only the sender's name is mentioned, but the receiving datacan be from any system. **send (P,** message) --- Send a message to process ***P***

  receive (id, message). Receive a message from any process

Disadvantages of direct communication – any changes in the identifier of a process, may have to change the identifier in the whole system (sender and receiver), where the messages are sent and received.

b) Indirect communication uses shared mailboxes, or ports.

A mailbox or port is used to send and receive messages. Mailbox is an object into which messages can be sent and received. It has a unique ID. Using this identifier messages are sent and received. Two processes can communicate only if they have a shared mailbox. The send and receive functions are –

- **send** (*A, message*) – send a message to mailbox A
- **receive** (*A, message*) – receive a message from mailbox A

Properties of communication link:

- A link is established between a pair of processes only if they have a shared mailbox
- A link may be associated with more than two processes
- Between each pair of communicating processes, there may be any number of links, each linkis associated with one mailbox.
- A mail box can be owned by the operating system. It must take steps to –
- create a new mailbox
- send and receive messages from mailbox
- delete mailboxes.

2. Synchronization

The send and receive messages can be implemented as either **blocking** or **non-blocking**.

Blocking (synchronous) send - sending process is blocked (waits) until the message isreceived by receiving process or the mailbox.

Non-blocking (asynchronous) send - sends the message and continues (does not wait)

Blocking (synchronous) receive - The receiving process is blocked until a message isavailable **Non-blocking (asynchronous) receive** - receives the message without block. Thereceived message may be a valid message or null.

3. Buffering

When messages are passed, a temporary queue is created. Such queue can be of three capacities: **Zero capacity** – The buffer size is zero (buffer does not exist). Messages are not stored in the queue. The senders must block until receivers accept the messages.

Bounded capacity- The queue is of fixed size(n). Senders must block if the queue is full.

After sending 'n' bytes the sender is blocked.

Unbounded capacity **- The queue is of infinite capacity. The sender never blocks.**

- **Multithreading Models**
  - **Many-to-One Model**
  - **One-to-One Model**
  - **Many-to-Many Model**

MULTITHREADING MODELS

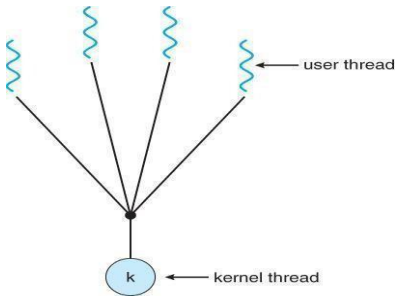
- Support for threads may be provided at either, 1. The user level, for **user threads** or

  2. By the kernel, for **kernel threads**.
- User-threads are supported above the kernel and are managed without kernel support. Kernel- threads are supported and managed directly by the OS.
- Three ways of establishing relationship between user-threads &kernel-threads:

  1. Many-to-one model

2. One-to-one model and 3. Many-to-many model.

Many-to-One Model



- Many user-level threads are mapped to one kernel thread.

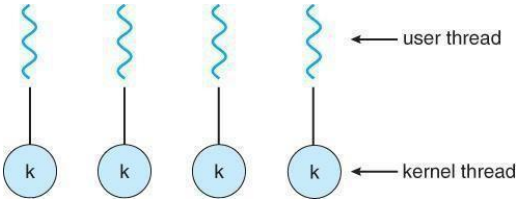
  Advantages:

  - Thread management is done by the thread library in user space, so it is efficient. *Disadvantages:*
  - The entire process will block if a thread makes a blocking system-call.
  - Multiple threads are unable to run in parallel on multiprocessors.
- For example:
  - Solaris green threads
  - GNU portable threads.

One-to-One Model

- Each user thread is mapped to a kernel thread.

  Advantages:



  - It provides more concurrency by allowing another thread to run when a threadmakes a blocking system-call.
  - Multiple threads can run in parallel on multiprocessors.

    Disadvantage:
  - Creating a user thread requires creating the corresponding kernel thread.
- For example:
  - Windows NT/XP/2000, Linux

Many-to-Many Model

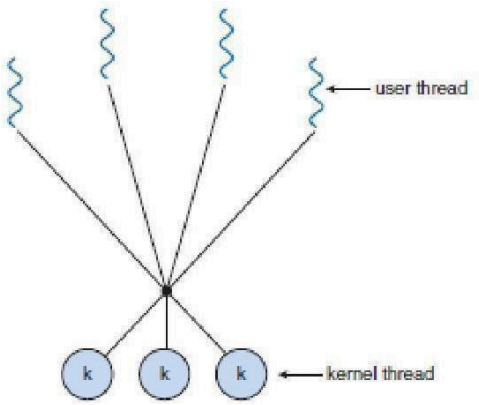
- Many user-level threads are multiplexed to a smaller number of kernel threads.

Advantages:

  - Developers can create as many user threads as necessary
  - The kernel threads can run in parallel on a multiprocessor.
  - When a thread performs a blocking system-call, kernel can schedule another threadfor execution.

Two Level Model

- A variation on the many-to-many model is the two level-model
- Similar to M:N, except that it allows a user thread to be bound to kernel thread.
- For example:
  - HP-UX
  - Tru64 UNIX



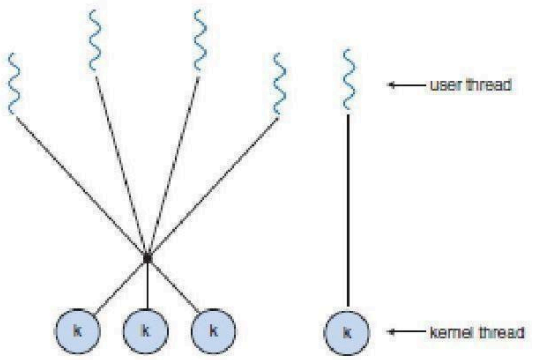


Fig: Many-to-many model Fig: Two-level model

- **Threading Issues**

THREADING ISSUES

fork() and exec() System-calls

- fork() is used to create a separate, duplicate process.
- If one thread in a program calls fork(),then
  1. Some systems duplicate all threads and
  2. Other systems duplicate only the thread that invoked the fork().
- If a thread invokes the exec(), the program specified in the parameter to exec() will replace the entire process including all threads. **Thread Cancellation**
- This is the task of terminating a thread before it has been completed.
- Target thread is the thread that is to be cancelled.
- Thread cancellation occurs in two different cases:
  1. ***Asynchronous cancellation***: One thread immediately terminates the target thread.
  2. ***Deferred cancellation***: The target thread periodically checks whether it should be terminated. **Signal Handling**
- In UNIX, a signal is used to notify a process that a particular event has occurred.
- All signals follow this pattern:
  1. A signal is generated by the occurrence of a certain event.
  2. A generated signal is delivered to a process.
  3. Once delivered, the signal must be handled.
- A signal handler is used to process signals.

- A signal may be received either synchronously or asynchronously, depending on the source.

  1. Synchronous signals

  - Delivered to the same process that performed the operation causing the signal.
  - E.g. illegal memory access and division by 0.

  2. Asynchronous signals

  - Generated by an event external to a running process.
  - E.g. user terminating a process with specific keystrokes<ctrl><c>.

🎬 Every signal can be handled by one of two possible handlers:

1. A Default Signal Handler

🕮 Run by the kernel when handling the signal.

2. A User-defined Signal Handler

🕮 Overrides the default signal handler.

- In ***single-threaded programs***, delivering signals is simple (since signals are always delivered to a process).
- In ***multithreaded programs***, delivering signals is more complex. Then, the following options exist:
  1. Deliver the signal to the thread to which the signal applies.
  2. Deliver the signal to every thread in process
  3. Deliver the signal to certain threads in the process.
  4. Assign a specific thread to receive all signals for the process.

THREAD POOLS

- The basic idea is to
  - create a no. of threads at process-startup and
  - place the threads into a pool (where they sit and wait for work).
- Procedure:
  1. When a server receives a request, it awakens a thread from the pool.
  2. If any thread is available, the request is passed to it for service.
  3. Once the service is completed, the thread returns to the pool.
- Advantages:
  - Servicing a request with an existing thread is usually faster than waiting tocreate a thread.
  - The pool limits the no. of threads that exist at any one point.
- No. of threads in the pool can be based on actors such as
  - no. of CPUs
  - amount of memory and
  - expected no. of concurrent client-requests.

THREAD SPECIFIC DATA

- Threads belonging to a process share the data of the process.
- this sharing of data provides one of the benefits of multithreaded programming.
- In some circumstances, each thread might need its own copy of certain data. We will call suchdata **thread-specific data**.
- For example, in a transaction-processing system, we might service each transaction in a separate thread.

- Furthermore, each transaction may be assigned a unique identifier. To associateeach thread with its unique identifier, we could use thread-specific data.

Differences

The differences between process and thread are,

	Process	**Thread**
1.	It is called heavyweight process.	It is called lightweight process.
2.	Process switching needs interface with OS.	Thread switching does not need interface with OS.
3.	Multiple processes use more resources than multiple threads.	Multiple threaded processes use fewer resources than multiple processes.
4.	In multiple process implementations each process executes same code but has its own memory and file resources.	All threads can share same set of open files.
5.	If one server process is blocked no other server process can execute until the first process unblocked.	While one server thread is blocked and waiting, second thread in the same task could run.
6.	In multiple processes each process operates independently of others.	One thread can read, write or even completely wipeout another threads stack.

Problems:- solved in class

(CPU scheduling algorithm, bankers algorithm)

(first 3 lab programs)